Sri Lanka Institute of Information Technology

IT3090 – Software Engineering Frameworks
Year 3, Semester 1- 2026

Mini Hackathon

Y3.S1.WE.AI.G15

Name	Student ID
Obesekara S.O.K.D.	IT24103866
Pehesara W.A.C.	IT24103684
Weerasooriya W.H.M.S.P.	IT24104192
Peiris W.S.V.	IT24103792

04.09.2026

Table of Contents

1. Problem Statement

The Sri Lanka Transport Board (SLTB) maintains an estimated 7,100 buses in the country. An appreciable number of buses – frequently more than 1,800 – are always out of operation at the depots owing to the following reasons:

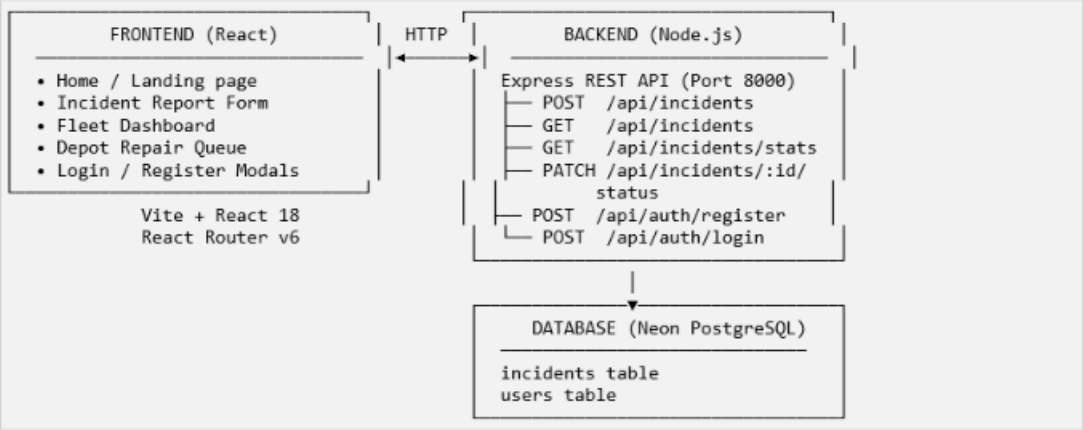
- Not tracking bus breakdowns - Lack of a digital tool to register and track bus breakdowns in real-time.
- Bottleneck maintenance processes - Workshop staff do not have insights into the backlog for priority-based decision making.
- Limited reporting of parts shortage - Are not raised above standard maintenance without severity analysis.
- Lack of service impact analysis - Lack of insight on whether the problem is critical or low impact.

All of which lead to service disruptions, low bus usage, and strand several thousand people daily. The primary issue in hand is the lack of a digital intake to resolution process linking drivers, workshop staff, and depot managers.

2. Solution Overview

SLTB DepotOps is a full-stack web application that digitizes the bus breakdown reporting and repair management workflow.

Architecture



Tech Stack

- Frontend - React 18, Vite, React Router v6, Vanilla CSS
- Backend - Node.js, Express 5, ES Modules
- Database - Neon PostgreSQL (cloud-hosted, serverless)
- Auth - JWT (jsonwebtoken), bcrypt (bcryptjs)
- Dev Tools - Nodemon, dotenv, CORS middleware

3. Functional Features

3.1 Incident Report (Driver / Field Staff)

- GPS Auto-location — Browser Geolocation API automatically captures latitude, longitude on form load.
- Structured breakdown form — Bus registration number, assigned depot, breakdown category, severity level, and free-text description.
- AI Severity Suggestion — "Auto-Suggest Severity" button analyses the description and pre-fills severity as Critical when hazardous conditions are detected.
- Dual-layer validation — Client-side (instant feedback) and server-side (authoritative). Server errors are mapped back to individual form fields.
- Real POST to database — Validated form data is persisted to Neon PostgreSQL via POST /api/incidents.

3.2 Fleet Dashboard (Admin)

- Live fleet metrics — Total incidents, Grounded / In Workshop count, and Repairs In Progress — all fetched in real time from GET /api/incidents/stats.
- Recent breakdown cards — The 5 most recently reported incidents with bus number, category, severity badge, depot, and relative timestamp (e.g., "14m ago").
- Error resilience — Graceful error banner if the API is unreachable.

3.3 Depot Repair Queue (Workshop Staff / Admin)

- Full incident table — All incidents loaded from GET /api/incidents, newest first.
- Live status updates — Dropdown per row sends PATCH /api/incidents/:id/status to the server. The confirmed status from the server response is reflected in the UI (no optimistic updates).

- Multi-filter support — Filter by Depot (Maharagama / Pettah / Meegoda), Status (Reported / In Workshop / In-Progress / Fixed), and free-text search by Bus Number or Ticket ID.
- Severity & Status badges — Colour-coded badges (Critical = red, High = orange, Low = grey).

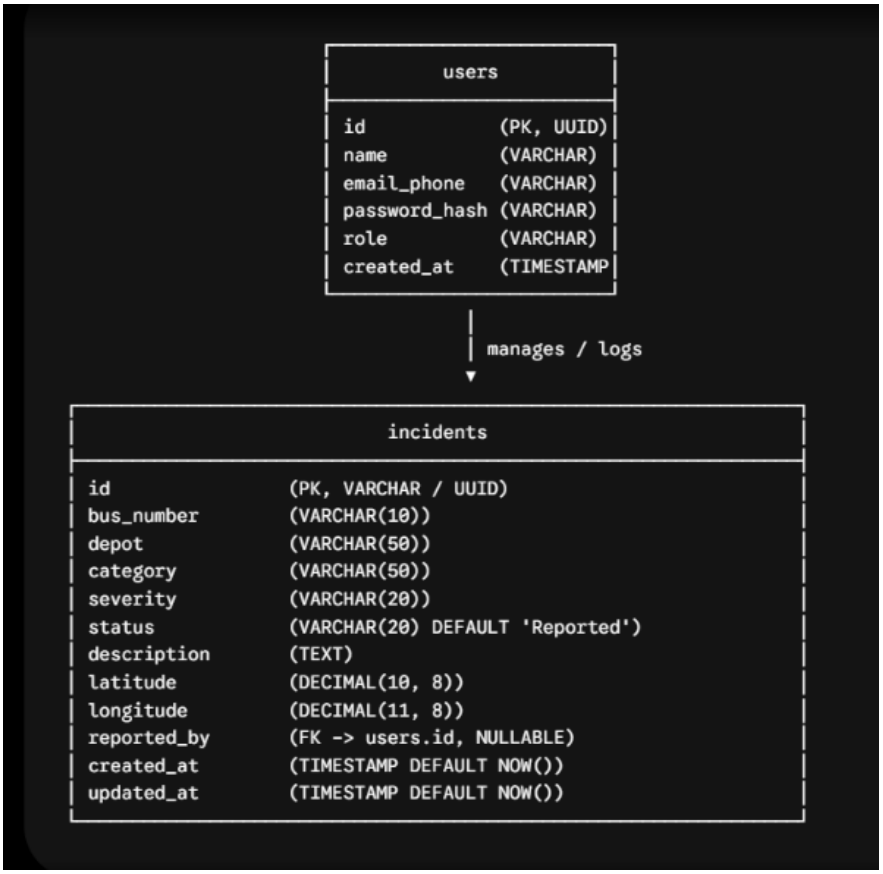
3.4 Authentication System

- User Registration — Name, phone or email, and hashed password stored to users table.
- User Login — Email or phone + password verified against bcrypt hash; JWT token returned.
- Protected routes — protectRoute middleware verifies Bearer tokens for secured endpoints.
- Modal-based UX — Login and Register presented as overlay modals on the Home page (no full-page redirect).

4. Input Form and Validation

Field	Type	Validation
GPS Location	Auto-filled	Browser navigator.geolocation — read-only display
Bus Registration No.	Text input	Required. Regex: /^[A-Z]{2,3}-\d{4}$/ (e.g. NB-4521, WP NA-1290)
Assigned Depot	Dropdown	Required. One of: Maharagama, Pettah, Meegoda
Breakdown Category	Dropdown	Required. One of: Engine Overheating, Brake Failure, Transmission, Electrical Issue, Body Damage
Severity Level	Dropdown	Required. One of: Low, High, Critical
Problem Description	Textarea	Optional. Max 1,000 characters

5. Data Handling



6. UI and Navigation

View Architecture & Routing

- / — **Landing Page / Overview:** Fleet summary cards, incident counters, fast action triggers, and login modal entry points.
- /report — **Incident Report View:** Driver fault submission form with geolocation readouts and AI severity assistant.
- /queue — **Depot Repair Queue:** Data grid containing multi-filtering options, ticket search, and real-time status transitions.
- /dashboard — **Fleet Analytics (Admin):** Visual metrics detailing grounded ratios, workshop workloads, and historical logs.

Design & Responsiveness

- Built using modular Vanilla CSS with fluid flexbox and grid containers.
- Adapts dynamically between desktop command center tables and touch-friendly mobile layouts optimized for workshop technicians.

7. Deployment

https://public-transport-one.vercel.app/

8. Github Repository

https://github.com/Krishmal2004/Public-Transport.git

9. Demo Video

https://drive.google.com/file/d/15lyIL1tYkbcQrpby-fHLST_c17nRz06V/view?usp=sharing

10. AI Prompt Log and Declaration

- We used ChatGPT, Claude and Gemini.

Prompt 1: Sri Lankan Vehicle Plate Pattern

"Provide a JavaScript regular expression that validates Sri Lankan vehicle registration numbers for both modern and older formats (e.g., 'WP NA-1290', 'NC-3341', '62-1234')."

Prompt 2: PostgreSQL Schema for Fleet Issues

"Create a PostgreSQL schema for tracking bus breakdown incidents with status tracking ('Reported', 'In Workshop', 'In-Progress', 'Fixed'), geolocation coordinates, foreign key user references, and automated timestamps."

11. Team Contributions

Obesekara S.O.K.D. (IT24103866) — Backend Lead & Cloud Deployment
Handled the complete database architecture and server deployment workflows. Configured the serverless Neon PostgreSQL database connection pooling with SSL support and managed cloud hosting environment configurations. Developed the core incident creation API (POST /api/incidents) with authoritative server-side validation middleware. Additionally, implemented the authentication system using bcrypt password hashing and JWT token issuance along with the protectRoute middleware to secure backend endpoints.

Weerasooriya W.H.M.S.P. (IT24104192) — Backend & API Architecture
Engineered the server-side API processing logic and analytics pipelines. Implemented the repair queue endpoints (GET /api/incidents) supporting dynamic multi-filtering and sorting, as well as the incident status update route (PATCH /api/incidents/:id/status) to manage live state changes. Created the fleet metrics aggregation API (GET /api/incidents/stats) to compute real-time operational statistics and implemented the rule-based logic to auto-suggest critical severity for high-risk safety hazards.

Pehesara W.A.C. (IT24103684) — Frontend Engineer (Incident Module)
Built the user-facing Incident Report module using React 18 and Vite. Implemented robust client-side input validation, including Sri Lankan vehicle registration regex pattern checks and instant inline error messages. Integrated the HTML5 Browser Geolocation API to auto-detect vehicle latitude and longitude coordinates. Also connected the reporting form to the backend REST API with server-side validation error mapping back to individual input fields.

Peiris W.S.V. (IT24103792) — Frontend Engineer (Dashboard & Queue)
Developed the core monitoring interfaces, including the Fleet Dashboard and the interactive Depot Repair Queue table. Implemented live queue features such as text search, multi-faceted filtering (depot, severity, status), and row-level status dropdowns with immediate UI state reflection. Designed modal-based login and registration overlays to preserve seamless single-page navigation, and styled the entire application using Vanilla CSS to ensure full responsiveness across desktop workstations and mobile screens.